Reviewer Pack — Minimal Hodge-theoretic Index and Frege Proof Depth Inequali...

Entry 10ae0a5c5991 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 11:12:22 UTC

Minimal Hodge-theoretic Index and Frege Proof Depth Inequality

Entry ID: `10ae0a5c5991` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 11:12:22 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Frege Proof Complexity

Statement:

> For every CNF φ with n variables, the minimal Hodge-theoretic index h(φ) of its associated algebraic variety X is linearly correlated with its Frege proof depth d(φ), such that h(φ) = Ω(d(φ)). Furthermore, for all CNFs, if d(φ) ≥ k, then h(φ) ≤ c · d(φ)^2, where c is a constant.

Rationale (proposer's reasoning):

> Hodge theory provides a framework for understanding the geometry of complex algebraic varieties. If the complexity of Frege proofs scales with the geometric properties of the associated variety, this could suggest a deeper connection between geometric and logical complexities.

Taxonomy category: `HodgeTheoreticIndexFregeProofDepthInequality` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b78aacc6b81cfd0a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For every CNF φ with n variables, if the minimal Hodge-theoretic index h(φ) is greater than or equal to Ω(d(φ)) for all seeds and the ratio of h(φ) to d(φ)^2 squared is less than or equal to a constant c when d(φ) ≥ k, then the conjecture is supported. Falsification occurs if any seed shows h(φ) < Ω(d(φ)) or h(φ)/d(φ)^2 > c for some φ with d(φ) ≥ k.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Hodge theory" AND "Frege proof complexity" AND minimal index"` - `"algebraic variety" [AND] "CNF" [AND] "Frege proof depth"` - `"Hodge-theoretic index" [AND] "linear correlation" [AND] "Frege proof depth"`

Top relevant hits considered: - [s2:f00d943cea727962981a17afc4650e9510d2008f] How a Math Class Can Be Two Places at Once: A Proposed Mathematical/Philosophical Collaboration

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(n):
            clause = [random.randint(1, n), -random.randint(1, n)]
            clauses.append(clause)
        return clauses

    def frege_proof_depth(cnf):
        # Simplified Frege proof depth simulation
        return len(cnf) * 2

    def hodge_theoretic_index(cnf):
        # Placeholder for Hodge-theoretic index calculation
        # For simplicity, we use the number of clauses as a proxy
        return len(cnf)

    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        cnf = generate_cnf(n)
        h_index = hodge_theoretic_index(cnf)
        d_depth = frege_proof_depth(cnf)
        results.append({"n": n, "h_index": h_index, "d_depth": d_depth})
```

```
    if not results:
        return {
            "metric_name": "Hodge-theoretic Index vs Frege Proof Depth",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "No instances generated"
        }

    h_values = [r["h_index"] for r in results]
    d_values = [r["d_depth"] for r in results]

    mean_h = sum(h_values) / len(h_values)
    mean_d = sum(d_values) / len(d_values)
    std_h = math.sqrt(sum((x - mean_h) ** 2 for x in h_values) / len(h_values))
    std_d = math.sqrt(sum((x - mean_d) ** 2 for x in d_values) / len(d_values))

    correlation = sum((h_values[i] - mean_h) * (d_values[i] - mean_d) for i in range(len(h_values)

    c = 1.0
    k = 5

    conjecture_holds = all(h >= d for h, d in zip(h_values, d_values)) and all(h <= c * d ** 2 fo

    return {
        "metric_name": "Hodge-theoretic Index vs Frege Proof Depth",
        "metric_value": correlation,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractio
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE no seeds tested")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
ture_holds': False, 'counterexample': ''}
TRIAL: {'metric_name': 'Hodge-theoretic Index vs Frege Proof Depth', 'metric_value': 1.00000000000000
TRIAL: {'metric_name': 'Hodge-theoretic Index vs Frege Proof Depth', 'metric_value': 1.00000000000000
TRIAL: {'metric_name': 'Hodge-theoretic Index vs Frege Proof Depth', 'metric_value': 1.00000000000000
TRIAL: {'metric_name': 'Hodge-theoretic Index vs Frege Proof Depth', 'metric_value': 1.00000000000000
TRIAL: {'metric_name': 'Hodge-theoretic Index vs Frege Proof Depth', 'metric_value': 1.00000000000000
TRIAL: {'metric_name': 'Hodge-theoretic Index vs Frege Proof Depth', 'metric_value': 1.00000000000000
TRIAL: {'metric_name': 'Hodge-theoretic Index vs Frege Proof Depth', 'metric_value': 1.00000000000000
TRIAL: {'metric_name': 'Hodge-theoretic Index vs Frege Proof Depth', 'metric_value': 1.00000000000000
TRIAL: {'metric_name': 'Hodge-theoretic Index vs Frege Proof Depth', 'metric_value': 1.00000000000000
TRIAL: {'metric_name': 'Hodge-theoretic Index vs Frege Proof Depth', 'metric_value': 1.00000000000000
RESULT: FALSIFIED counterexample="" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The test uses the number of clauses as a proxy for the Hodge-theoretic index, which is not correct. The Hodge-theoretic index is a complex algebraic geometric property that cannot be accurately represented by the count of clauses in a CNF. Additionally, the Frege proof depth calculation is simplified and does not reflect the actual complexity of proving a clause.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test results indicate that the conjecture does not hold for at least one seed (first_failing_seed=11), as h(φ) < Ω(d(φ)) or h(φ)/d(φ)^2 > c for so | next: Further investigation is needed to determine the exact relationship between Hodge-theoretic index and Frege proof depth. Consider using more accurate methods for calculating these metrics and retesting the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16058
2	preregistration	ollama_remote	glm4:latest	0	0	10516
3	novelty	ollama_remote	glm4:latest	0	0	12265
4	novelty	ollama_remote	glm4:latest	0	0	8902
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	38651
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10526
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9254
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18554
9	critic	ollama_remote	glm4:latest	0	0	50849

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
10	judge	ollama_remote	glm4:latest	0	0	10015

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 185591 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/10ae0a5c5991.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/10ae0a5c5991.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/10ae0a5c5991.tar.gz` (if generated)